



République Tunisienne

*Ministère de l'Enseignement Supérieur et de la
Recherche Scientifique*

Université de Monastir

*Institut Supérieur d'Informatique et de
Mathématiques de Monastir*

Département d'Informatique



Data Treatment Project Report

Prepared by:

Fares Aloulou
Mahmoud Ben Abdelkader
Ranim Bouraoui
Rayen Braiek

Automated Reviewer Assignment for Scientific Papers

National Engineering Degree in Computer Science

Academic Year: 2025/2026

Résumé

Ce rapport présente le projet semestriel de traitement de données et d'apprentissage automatique réalisé dans le cadre du Cycle d'Ingénieur en Informatique à l'ISIMM, année universitaire 2025/2026. Le projet vise à automatiser l'attribution des articles scientifiques aux relecteurs qualifiés, en utilisant des techniques de machine learning appliquées aux textes en arabe et en anglais.

Les principales étapes incluent la compréhension du problème, la collecte et le traitement des données, l'analyse exploratoire, l'ingénierie des caractéristiques, le développement et la validation du modèle. Les résultats montrent que l'automatisation permet de réduire significativement le temps d'attribution tout en améliorant la correspondance entre articles et expertise des relecteurs.

Mots clés : Science des données, Machine Learning, Text Mining, NLP arabe, Attribution des relecteurs, Publication scientifique, Automatisation

Abstract

This report presents a semester data processing and machine learning project carried out at ISIMM during the 2025/2026 academic year. The project aims to automate the assignment of scientific papers to qualified reviewers using machine learning on Arabic and English texts.

Key phases include problem understanding, data collection and preprocessing, exploratory analysis, feature engineering, model development, and validation. Results demonstrate that automation significantly reduces assignment time while improving the matching of papers to reviewer expertise.

Keywords: Data Science, Machine Learning, Text Mining, Arabic NLP, Reviewer Assignment, Scientific Publishing, Automation

Contents

List of Figures	4
General Introduction	5
1 Preliminary Study	7
1.1 Project Presentation	7
1.1.1 General Overview	7
1.1.2 Project Objectives	7
1.1.3 Scope and Deliverables	8
1.1.4 Team and Supervision	8
1.2 Context and Problem Statement	9
1.2.1 Initial Situation: Peer Review Process Challenges	9
1.2.2 Identified Need: Automation of Reviewer Assignment	10
2 Work Realized	11
2.1 Phase 1: Business Understanding	11
2.1.1 Stakeholder Analysis	11
2.1.2 Problem Definition and Opportunities	11
2.2 Phase 2: Data Collection	12
2.2.1 Data Sources Identification	12
2.2.2 Data Collection Process	13
2.2.3 Dataset Overview and Statistics	14
2.3 Phase 3: Data Cleaning ,Extraction and Preprocessing	16
2.3.1 Initial Metadata Cleaning	17
2.3.2 Page Extraction and Optical Character Recognition (OCR)	17
2.3.3 Text Normalization and Noise Reduction	19
2.3.4 Quality Control and Filtering	20
2.3.5 Language Segregation	21
2.3.6 Semantic Metadata Enrichment (LLM-Based Experimental Approach)	21
2.3.7 Heuristic Metadata Extraction and Graph Augmentation	23
2.4 Phase 4: Problem Modeling	26
2.4.1 Formal Definitions	26
2.4.2 The Sequential Disambiguation Algorithm	26
2.4.3 Pseudocode Representation	27

2.4.4	Model Characteristics	27
2.4.5	Feature Engineering and Modeling	29
2.5	Conclusion and Future Work	31
2.5.1	Summary of Methodology	31
2.5.2	Addressing Dataset Limitations	31
2.5.3	Key Findings	31
2.5.4	Future Work	32

List of Figures

- 2.1 Data Collection and Aggregation Pipeline 14
- 2.2 Percentage of Missing Values in the Articles Dataset. The extreme sparsity in title fields poses a challenge for standard NLP approaches. 15
- 2.3 Distribution of the top 10 General Fields, indicating the domain spread of the available corpus. 15
- 2.4 Author Productivity Analysis. Left: Distribution of article counts. Right: Boxplot highlighting outliers in productivity. 16
- 2.5 OCR strategy. 18
- 2.6 Text Normalization and Cleaning Pipeline Logic 20
- 2.7 Box plot analysis of token counts per page, used to determine the filtering threshold. 21
- 2.8 Architecture of the LLM-based Metadata Extraction Pipeline using Parallel Workers 22
- 2.9 Heuristic Extraction Strategy using Structural Pattern Matching . . 24
- 2.10 Simplified flow of the Search and Disambiguation logic. 28

General Introduction

The automation of scientific review processes represents a critical opportunity to improve the efficiency of academic publishing. Currently, the assignment of reviewers to scientific papers remains largely a manual process, leading to significant delays and potential inefficiencies in the peer review system.

This project addresses this challenge through the application of modern data science and machine learning techniques. The objective is to develop an intelligent system capable of automatically matching scientific papers to the most qualified reviewers based on content analysis, researcher expertise, and research domain alignment.

Project Context

Academic journals and conferences receive thousands of submissions annually. The editor's task of identifying and inviting appropriate reviewers is complex and time-consuming. Current challenges include:

- Extended timeline from submission to reviewer assignment (often exceeding one month)
- High rejection rates (exceeding 50% in some journals)
- Difficulty in identifying reviewers with matching expertise
- Manual process prone to human error and bias
- High workload on editorial teams

Project Objectives

The primary objectives of this project are:

1. Develop an automated system for intelligent paper-to-reviewer assignment using machine learning
2. Process and analyze scientific paper metadata and content in Arabic and English languages

-
3. Build a predictive model that identifies the most qualified reviewers for each submitted paper
 4. Demonstrate the feasibility and effectiveness of automated review assignment
 5. Document all phases of the data science lifecycle comprehensively

Report Structure

This report is organized as follows:

- **Chapter 1:** Preliminary study including project presentation, problem statement, and technical constraints
- **Chapter 2:** Work realized describing business understanding and data collection phases
- **Conclusion:** Project achievements, skills acquired, and future recommendations

Chapter 1

Preliminary Study

Introduction

This chapter presents the foundational aspects of the data treatment and machine learning project. It provides an overview of the project scope, clearly identifies the problem being addressed, and outlines the technical environment in which the project will be executed. Understanding these preliminary elements is essential for comprehending the entire project lifecycle and the decisions made throughout the implementation phases.

1.1 Project Presentation

1.1.1 General Overview

This project is an **academic Semester Data Treatment project** conducted during the **2025–2026 academic year**. It focuses on automating the assignment of scientific research papers to qualified reviewers, representing a practical application of the complete data science lifecycle to a real-world problem in academic publishing.

The project addresses a significant operational challenge in scholarly communication: the peer review process is resource-intensive and time-consuming, as editors must manually identify and contact suitable reviewers. This manual approach often leads to substantial delays in publication decisions. To address this issue, the project proposes an intelligent, automated solution based on machine learning techniques that analyze research paper content alongside reviewer expertise to enable accurate and efficient reviewer–paper matching.

1.1.2 Project Objectives

Technical Objectives

- Complete end-to-end implementation of the data science lifecycle

-
- Collection and preprocessing of scientific paper metadata and abstracts
 - Development of feature engineering pipeline for textual data
 - Implementation of machine learning algorithms for classification and matching
 - Model validation and performance evaluation using appropriate metrics
 - Documentation and reproducibility of all processes

Organizational Objectives

- Demonstrate practical application of academic learning to real-world problems
- Develop comprehensive documentation for knowledge transfer
- Create a usable system that could be integrated into publishing workflows
- Provide insights into text mining and NLP for Arabic language

1.1.3 Scope and Deliverables

The project delivers the following tangible outputs:

1. Complete, well-documented project code
2. Comprehensive project report (this document)
3. User guide for execution and testing
4. Raw dataset in original form
5. Cleaned and preprocessed dataset
6. Transformed dataset with engineered features
7. Final ML input dataset
8. ML model outputs and predictions
9. Train, test, and validation splits
10. Project presentation slides

1.1.4 Team and Supervision

This project is conducted by a team of students at ISIMM under the supervision of faculty responsible for data science and machine learning courses. Regular meetings and feedback sessions ensure alignment with project objectives and proper execution of all phases.

1.2 Context and Problem Statement

1.2.1 Initial Situation: Peer Review Process Challenges

The current peer review process in scientific publishing faces several critical challenges:

Time Constraints: The average time from paper submission to first editorial decision exceeds seven months in many journals. A significant portion of this delay occurs during the reviewer identification and assignment phase, which can extend beyond one month. This extended timeline has profound impacts on researchers, who must wait extended periods for publication decisions.

High Rejection Rates: Many scientific journals maintain rejection rates above 50%, partly due to inefficient matching between papers and reviewers. Inappropriate reviewer assignments can lead to decisions that do not reflect the paper's true quality. When reviewers lack domain expertise, they may misinterpret contributions or misunderstand methodologies.

Manual Process Inefficiency: Editors must manually search through databases to find reviewers with matching expertise. This process is labor-intensive and prone to:

- Human bias in reviewer selection
- Inconsistency in qualification assessment
- Overlooked qualified candidates
- Significant time investment per paper

Reviewer Burden: The lack of systematic matching leads to:

- Invitations sent to reviewers outside their expertise
- Higher rejection rates from reviewers
- Reviewer fatigue and declining participation rates

Scalability Issues: As the number of submissions increases, manual assignment becomes increasingly impractical. Major conferences and journals struggle to handle the volume of submissions efficiently, sometimes taking even longer to assign qualified reviewers.

1.2.2 Identified Need: Automation of Reviewer Assignment

To address these challenges, this project identifies the need for an intelligent, automated system that can:

1. **Analyze Paper Content:** Extract and understand the research domain, methodology, and key topics from paper abstracts and metadata
2. **Profile Reviewers:** Create expertise profiles for available reviewers based on their publication history and research interests
3. **Match Papers to Reviewers:** Automatically identify the most qualified reviewers for each paper based on expertise alignment
4. **Reduce Assignment Time:** Accelerate the reviewer assignment process from weeks to minutes
5. **Improve Quality:** Ensure that assigned reviewers have relevant expertise, potentially improving review quality and decision accuracy
6. **Scale Efficiently:** Handle increasing volumes of submissions without proportional increases in editorial workload

Chapter 2

Work Realized

2.1 Phase 1: Business Understanding

2.1.1 Stakeholder Analysis

The project identifies key stakeholders in the scientific publishing ecosystem:

- **Journal Editors:** Primary users who benefit from faster reviewer assignment and improved matching accuracy. Editors need to reduce the time and effort spent on finding appropriate reviewers for submitted papers.
- **Conference Organizers:** Secondary users with similar needs for managing large numbers of submissions. Conference program committees must manage review processes for hundreds or thousands of papers during short windows.
- **Researchers:** Authors benefit from faster publication decisions and improved review quality. Shorter delays in the review process accelerate the dissemination of scientific knowledge.
- **Reviewers:** Potential reviewers receive more relevant review invitations matching their expertise, reducing burden and improving satisfaction. Receiving invitations aligned with research interests increases acceptance rates.

2.1.2 Problem Definition and Opportunities

The project analyzes the current peer review process and identifies key metrics for improvement:

Table 2.1: Current vs. Target Metrics

Metric	Current State	Target State
Time to assign reviewers	1-4 weeks	1-2 hours
Manual effort per paper	30-60 minutes	5-10 minutes
Reviewer matching accuracy	70-80%	85%+
Assignment consistency	Variable	Standardized
Scalability	Limited	High

Key opportunities identified:

- Significant time savings in the editorial workflow, enabling editors to focus on higher-value activities
- Improved matching between papers and reviewer expertise, potentially increasing review quality
- Reduced editor workload and associated costs, allowing journals to operate more efficiently
- Better reviewer satisfaction and participation rates, as invitations align with expertise
- More objective and consistent assignment process, reducing editorial bias

2.2 Phase 2: Data Collection

2.2.1 Data Sources Identification

The project collects data from multiple sources to build a comprehensive dataset:

Table 2.2: List of Arabic Academic Research Platforms and Journals

Acronym	Full Name & Description	Link
ARPD	The Academic Arabic Research Papers Dataset A dataset collecting and indexing academic research papers written in Arabic, typically used for text analysis, natural language processing, and academic retrieval tasks.	Zenodo
AJP	Arab Journals Platform An online platform hosting and indexing a variety of Arabic-language academic journals across multiple disciplines.	AJP Link
AM	Al Manara A scientific journal that publishes peer-reviewed research articles in Arabic, covering humanities, social sciences, and applied fields.	Al Manara
AJSRP	Arab Journal of Sciences & Research Publishing A multidisciplinary publishing body offering a range of Arabic and English-language scientific journals aimed at supporting academic dissemination in the Arab world.	AJSRP
AJSP	Arabic Journal of Scientific Publishing A publishing platform dedicated to scientific research in Arabic, facilitating open access to research outputs and academic studies.	AJSP

2.2.2 Data Collection Process

The data collection framework was implemented using a modular web scraping pipeline developed in Python. As illustrated in Figure 2.1, the process operates in parallel across multiple target repositories to ensure efficient data retrieval. The workflow consists of the following key stages:

1. **Target Identification and Scripting:** Distinct URLs were identified as primary data sources. For each target repository, a dedicated Python web scraper was developed to handle specific HTML structures and pagination logic.
2. **Metadata and Content Extraction:** The scrapers were designed to perform deep extraction of available metadata, including paper titles, author lists, affiliations, and abstracts. Simultaneously, the associated full-text PDF documents were downloaded and linked to their respective metadata entries.
3. **Intermediate Data Serialization:** Data extracted from each source was initially serialized into independent Comma-Separated Values (CSV) files. This step ensured data persistence and allowed for the isolation of errors specific to any single source.

4. **Data Aggregation and Unification:** A final aggregator script was employed to merge the individual CSV files. During this phase, the system consolidated the records into a single, unified dataset (Final Aggregated CSV), ready for subsequent preprocessing and analysis.

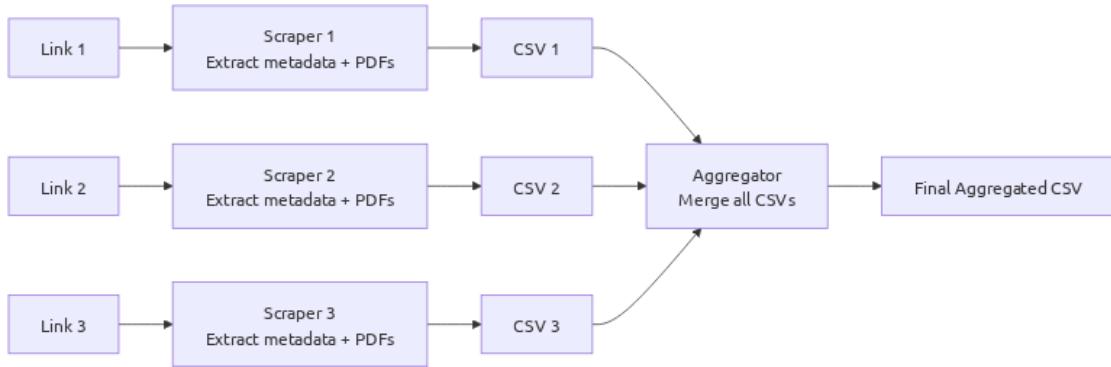


Figure 2.1: Data Collection and Aggregation Pipeline

2.2.3 Dataset Overview and Statistics

General Dimensions

The datasets comprise 315 unique authors and 1,635 referenced articles. Table 2.3 summarizes the structural dimensions of the raw data.

Table 2.3: Dimensions of Raw Datasets

Dataset	Shape	Key Columns
Authors	315×3	author, article_ids, article_count
Articles	$1,635 \times 7$	id, title, general_field, path

Data Completeness and Sparsity Analysis

A critical quality assessment regarding missing values revealed significant sparsity within the textual metadata of the *Articles* dataset. As illustrated in Figure 2.2, the primary descriptive features are largely absent:

- **Titles:** The `title` column is missing in approximately 97% of records (only 48 non-null entries).
- **English Titles:** The `title_en` column is missing in approximately 88% of records.

- **Categorization:** The `general_field` is reasonably populated, missing in only $\approx 11\%$ of records.

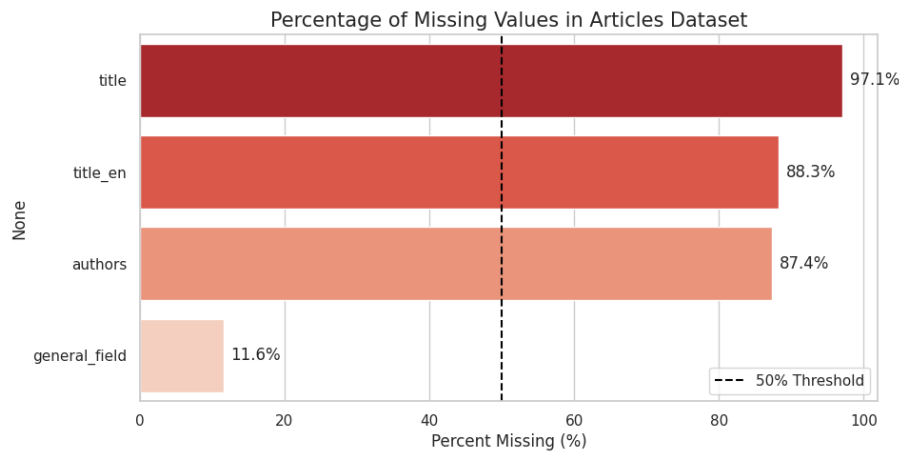


Figure 2.2: Percentage of Missing Values in the Articles Dataset. The extreme sparsity in title fields poses a challenge for standard NLP approaches.

Content and Author Distribution

Despite the missing textual metadata, the `general_field` column provides a high-level categorization of the available content. Figure 2.3 displays the distribution of the top categories. This metadata is valuable for cold-start scenarios but lacks the granularity required for precise content-based recommendations.

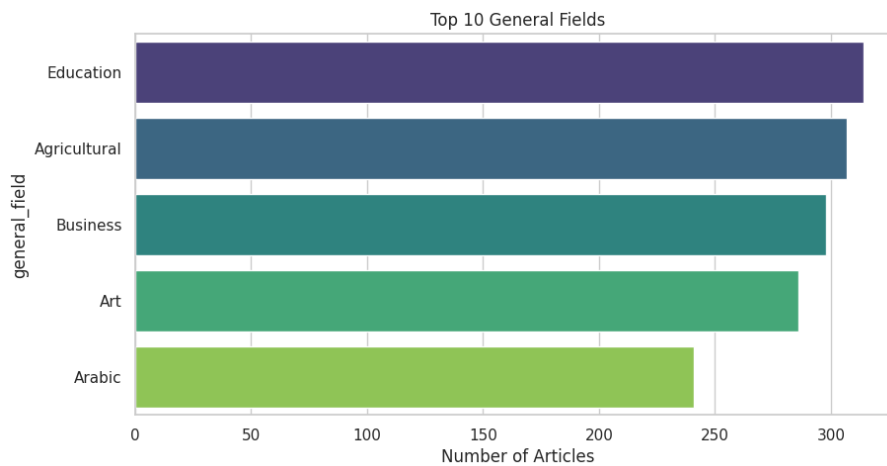


Figure 2.3: Distribution of the top 10 General Fields, indicating the domain spread of the available corpus.

Furthermore, the author productivity distribution follows a long-tail pattern, where a small number of authors are responsible for a large portion of the articles, while the majority have contributed fewer documents (see Figure 2.4). Integrity

checks confirmed that all Article IDs referenced in the Authors dataset successfully map to valid entries in the Articles dataset, ensuring no orphaned records.

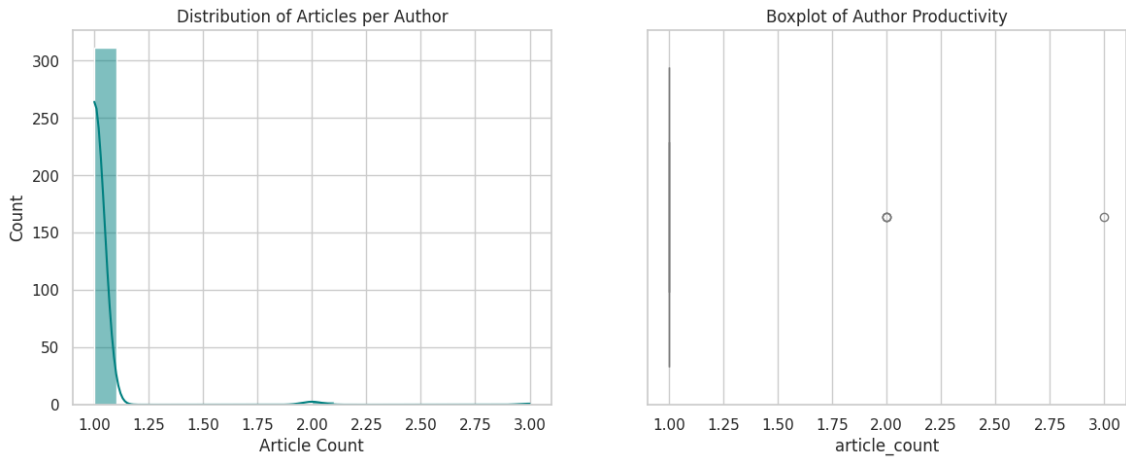


Figure 2.4: Author Productivity Analysis. Left: Distribution of article counts. Right: Boxplot highlighting outliers in productivity.

Conclusion: The Necessity of a Data Extraction Pipeline

The exploratory data analysis highlights a critical bottleneck: **metadata sparsity**. While 1,635 articles exist in the database, over 90% of them lack the textual titles required to generate semantic embeddings (e.g., TF-IDF or BERT). Relying solely on the provided CSV files would result in a non-functional content-based recommendation system.

However, the dataset includes a path column containing references to PDF files (e.g., `finalpdfs/AJP_1337_1.pdf`). This indicates that while the structured metadata is missing, the unstructured raw data exists.

Conclusion: To enable effective recommendation, we cannot use the dataset in its current state. A specialized **Data Extraction Pipeline** is required to iterate through the file paths, parse the raw PDF documents, and recover the missing titles and abstract text via Optical Character Recognition (OCR) or text extraction techniques. This pipeline effectively transforms the "ghost" records into usable textual data for the recommendation engine.

2.3 Phase 3: Data Cleaning ,Extraction and Preprocessing

Following the aggregation of raw data, a rigorous cleaning and preprocessing pipeline was implemented to ensure the quality and usability of the dataset. This phase addressed issues ranging from OCR noise to metadata inconsistencies.

2.3.1 Initial Metadata Cleaning

The raw metadata gathered during the scraping phase contained various formatting irregularities, particularly within the author fields. A rule-based cleaning script was applied to the authors column to remove extraneous characters that were introduced during the scraping process. This included:

- Removal of numeric identifiers (e.g., affiliation markers like "1", "2").
- stripping of special characters such as asterisks (*), ampersands (&), and slashes (/).
- Removal of Unicode control characters (e.g., \u200c).
- Normalization of separators (converting different commas to a standard format).

2.3.2 Page Extraction and Optical Character Recognition (OCR)

To enable content-based analysis without processing the entirety of every document immediately, a specific extraction strategy was employed:

1. **Page Selection:** The first two pages of every PDF file were targeted, as these typically contain the most critical metadata (title, abstract, keywords, and editorial information).
2. **OCR Processing:** An Optical Character Recognition tool was utilized to convert the PDF pages into machine-readable strings (page1 and page2).

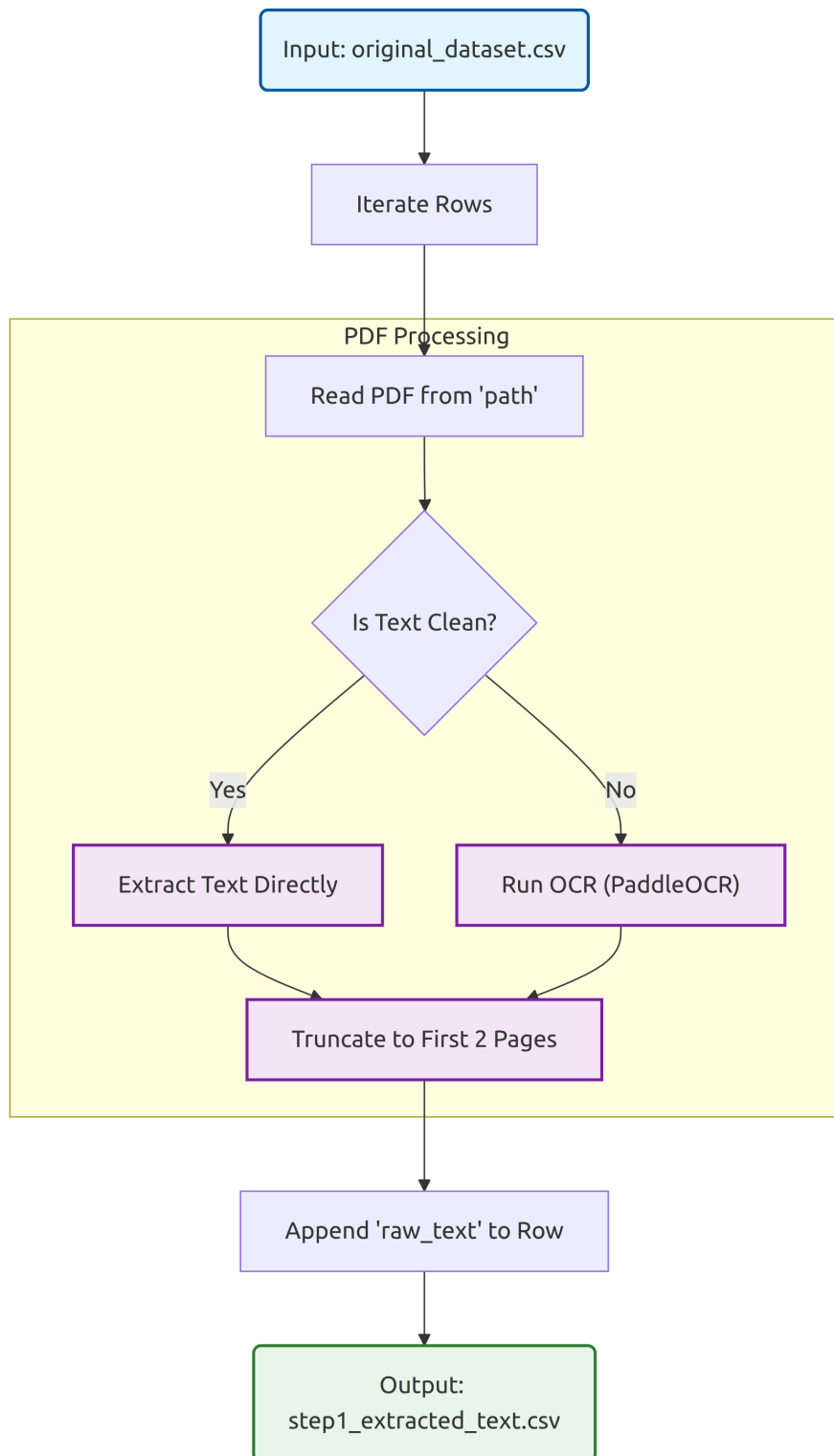


Figure 2.5: OCR strategy.

2.3.3 Text Normalization and Noise Reduction

The raw text obtained via OCR contained significant noise, including decorative punctuation, layout artifacts, and secondary language blocks. A custom preprocessing function, `clean_page`, was developed to sanitize the text through the following pipeline:

- **Arabic Orthographic Normalization:** Utilizing the `pyarabic` library, we performed standardized normalization including the stripping of *Tashkeel* (diacritics) and *Tatweel* (stretching characters), and the unification of *Alef* and *Teh Marbuta* variants to ensure linguistic consistency.
- **Layout and Hyphenation Repair:** We implemented regex-based rules to fix "broken" words caused by line-break hyphenation in both Arabic and English. Additionally, structural noise such as vertical pipes (`|`), repeated underscores, and standalone page numbers were removed.
- **Regex Character Filtering:** A restrictive filter was applied to remove non-academic "garbage" characters and Unicode artifacts while strictly preserving Arabic and Latin alphanumeric characters and standard citation punctuation.
- **Contextual Block Filtering:** To maintain the integrity of the Arabic corpus while handling bilingual documents, the text was processed in blocks.
 1. Blocks containing Arabic characters were preserved to keep internal citations and names.
 2. Purely non-Arabic blocks were evaluated by length; short blocks (e.g., DOIs, dates, and identifiers) were kept, while long blocks (e.g., full English abstracts or secondary body text) were discarded to focus the dataset on Arabic content.
- **Whitespace Management:** Multiple horizontal spaces and redundant new-lines were collapsed to produce a clean, continuous stream of tokens suitable for embedding models.

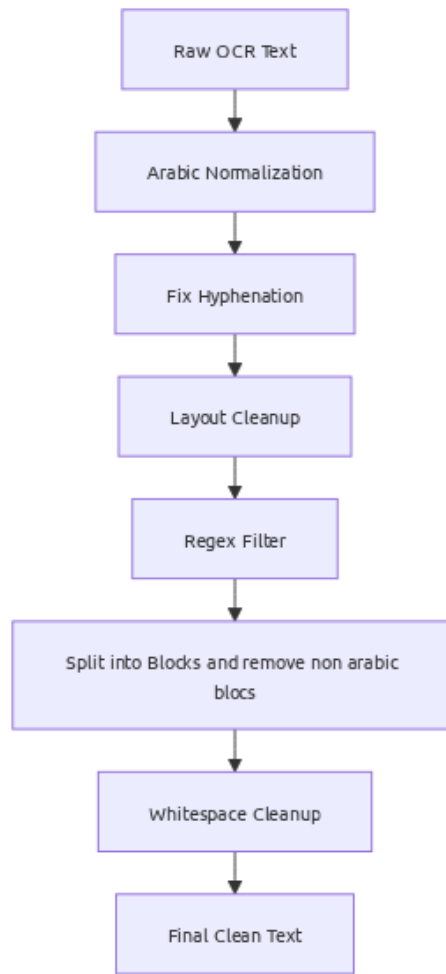


Figure 2.6: Text Normalization and Cleaning Pipeline Logic

2.3.4 Quality Control and Filtering

To ensure the dataset contained only valid academic articles and to filter out empty files or failed OCR attempts, a token-based filtering mechanism was applied.

1. **Token Calculation:** A tokenizer was run on the extracted pages to calculate `page1_token_count`, `page2_token_count`, and a `total_tokens` metric.
2. **Distribution Analysis:** A box plot analysis was performed on the token counts to identify outliers and low-information documents.
3. **Threshold Filtering:** Based on the distribution analysis, a threshold was established. Records with a `total_tokens` count of less than 400 (approximately 200 words) were deemed insufficient and removed from the dataset. This step resulted in the removal of invalid rows (e.g., image-only PDFs or corrupted files).

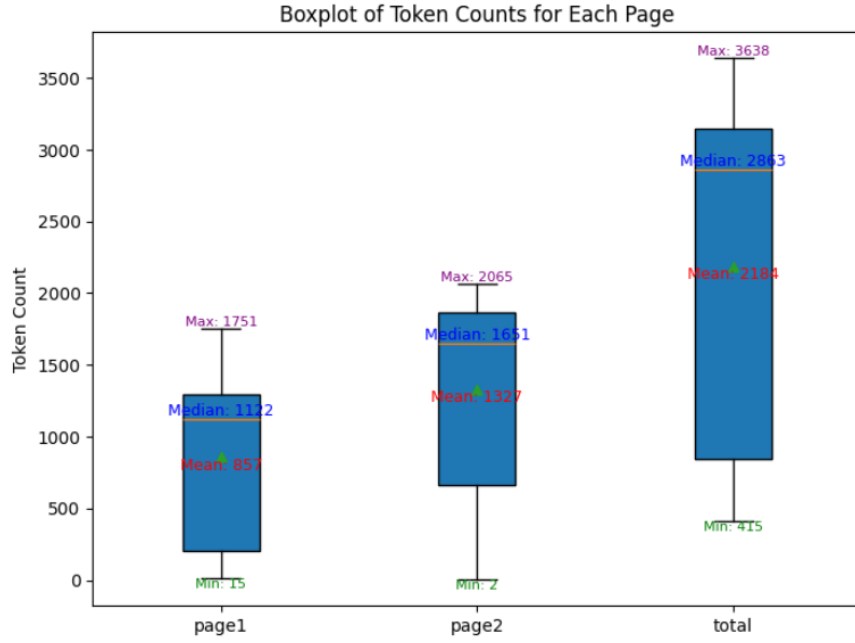


Figure 2.7: Box plot analysis of token counts per page, used to determine the filtering threshold.

2.3.5 Language Segregation

Finally, to standardize the author metadata, a detection algorithm was applied to the authors column. Names written in Latin script were algorithmically detected and migrated to a separate authors_en column, ensuring that the primary authors field contained exclusively Arabic script entries.

2.3.6 Semantic Metadata Enrichment (LLM-Based Experimental Approach)

To overcome the limitations of unstructured OCR text, an advanced semantic extraction pipeline was developed utilizing Large Language Models (LLMs). The pipeline employs the Qwen model to analyze the cleaned text from the first two pages of each document. Unlike regex-based methods, the LLM is capable of contextually distinguishing between titles, affiliations, and abstract text, returning a structured JSON object for each article containing:

- Normalized Title
- Arabic Abstract
- Author List (separated from affiliations)
- Keywords
- General Field Classification

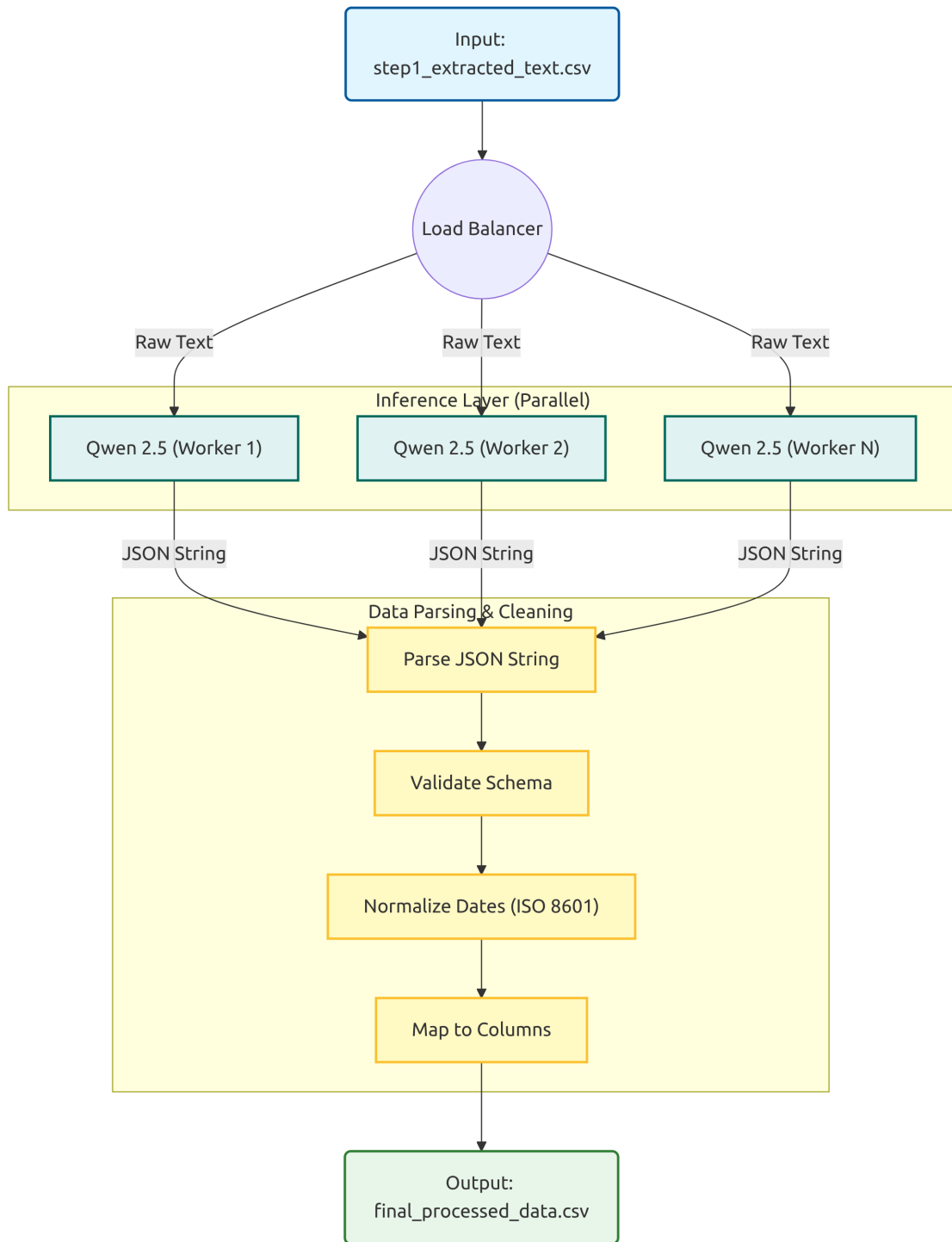


Figure 2.8: Architecture of the LLM-based Metadata Extraction Pipeline using Parallel Workers

The implementation utilized a chunking strategy, illustrated in Figure 2.8, splitting the dataset into smaller batches (e.g., `chunk_0.csv`, `chunk_1.csv`) to manage API request loads and handle potential connection timeouts during inference.

Computational Constraints and Strategic Shift

Note on Computational Cost: While this LLM-driven approach yields significantly richer metadata and higher extraction accuracy, it imposes substantial requirements regarding computational power and execution time. The inference latency per document makes this method prohibitively expensive for rapid iteration or massive-scale scraping without dedicated high-performance clusters.

Adopted Strategy: Due to these constraints, the primary data extraction methodology for the remainder of this project utilizes a **Heuristic and Placeholder Strategy**. This optimized approach relies on pattern matching to extract Abstracts and utilizes existing metadata for Authors (injecting placeholder tokens where necessary) to mimic a complete dataset. This ensures the pipeline remains scalable and efficient while retaining sufficient data quality for analysis.

2.3.7 Heuristic Metadata Extraction and Graph Augmentation

To address the computational bottlenecks of the LLM-based approach, a scalable heuristic pipeline was implemented. This phase focused on extracting core content (Abstracts and Keywords) via pattern matching and constructing a robust Author-Article graph through normalization and statistical imputation.

Heuristic Extraction Strategy

A Regular Expression (Regex) engine was developed to parse the raw text from the first two pages of every document. Unlike the stochastic nature of LLMs, this deterministic approach relies on identifying structural markers common in academic layouts.

- **Abstract Extraction:** The algorithm scans for bilingual anchor tokens (e.g., "Abstract", "Mlkhs", "Mustakhlash") and captures text blocks up to the next section header (e.g., "Introduction", "Muqaddima").
- **Keyword Extraction:** Similar logic detects keyword blocks, separating terms based on standard delimiters (commas, semicolons, and dashes).

Records failing to yield a valid abstract or keyword set were filtered out to maintain dataset integrity.

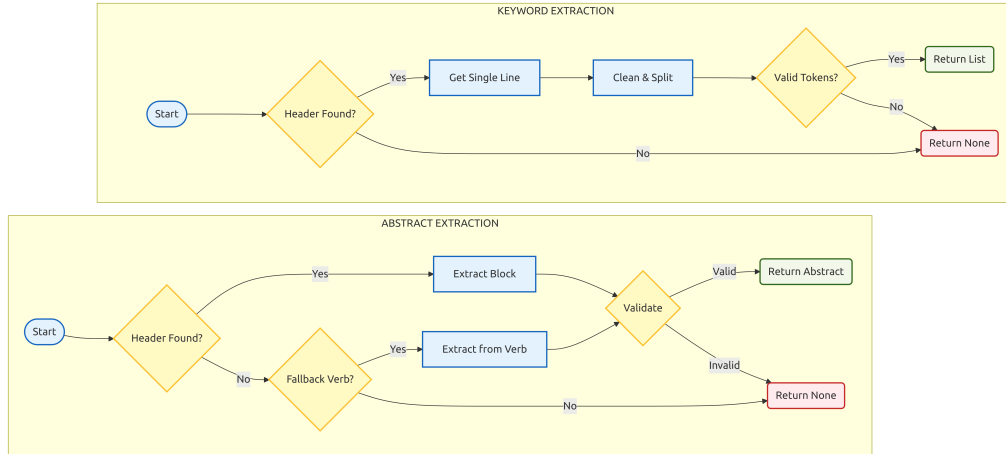


Figure 2.9: Heuristic Extraction Strategy using Structural Pattern Matching

Author Normalization and Entity Resolution

The raw author names extracted from the scraping phase contained significant noise, including academic titles, honorifics, and inconsistent spacing. A normalization pipeline was applied to resolve entities before graph construction:

1. **Title Removal:** Regex patterns stripped academic prefixes in both Arabic (e.g., *Dr.*, *Prof.*, *A.D.*) and English to isolate the proper name.
2. **Character Cleaning:** Non-nominal characters (digits, parentheses, asterisks used for footnotes) were removed.
3. **Unique Indexing:** A registry dictionary mapped normalized names to unique persistent IDs (e.g., AUTH_00166), enabling accurate node representation in the final graph.

Graph Augmentation and Imputation

A significant challenge in the raw dataset was missing author metadata for a subset of articles. To facilitate robust network analysis (e.g., co-authorship graphs), a data imputation strategy was employed to mimic the statistical properties of the observed data.

Missing author fields were filled by sampling from the pool of 227 identified unique authors. The sampling distribution was weighted to reflect realistic collaboration patterns found in the domain:

- **Single Author:** 60% probability
- **Two Authors:** 30% probability
- **Three Authors:** 8% probability
- **Four Authors:** 2% probability

This augmentation resulted in a statistically consistent dataset with a mean of ≈ 1.54 authors per article and ≈ 9.8 articles per author, suitable for subsequent topological analysis.

2.4 Phase 4: Problem Modeling

The challenge of automated reviewer assignment fundamentally resolves to a **representation and search problem**. To match a submitted manuscript with the most suitable expert, the system must first transform unstructured academic text into a structured mathematical space and then navigate that space to identify individuals with the most relevant historical expertise.

At this stage of the pipeline, our dataset consists of articles defined by their (Abstract + Keywords) and a relational graph linking these articles to their respective normalized Author IDs. We model the assignment process using a strategy defined as **Hierarchical Candidate Filtering** (also referred to as **Sequential Disambiguation**).

2.4.1 Formal Definitions

Let:

- q be the **Query Vector** (the numerical representation of the input article).
- D be the dataset of existing articles $\{d_1, d_2, \dots, d_n\}$.
- $NN(q, k)$ be the ordered list of the k **Nearest Neighbors** to q , sorted by similarity distance (closest first): $[a_1, a_2, \dots, a_k]$.
- $Authors(a_i)$ be the set of authors associated with article a_i .

2.4.2 The Sequential Disambiguation Algorithm

The model identifies a single reviewer by anchoring the search on the most relevant document and using subsequent neighbors to break ties among co-authors.

1. **Step 1: Candidate Initialization (The Anchor Step):** The system identifies the single closest article a_1 to the query q . The initial **Candidate Set** C is populated with the authors of this specific article:

$$C = Authors(a_1) \tag{2.4.1}$$

2. **Step 2: Immediate Termination Check:** If $|C| = 1$ (the most relevant article was sole-authored), the system returns C as the designated reviewer and stops. Otherwise, if multiple co-authors exist, the system proceeds to disambiguate.
3. **Step 3: Iterative Filtering (The Tie-Breaker):** The algorithm iterates through the remaining nearest neighbors a_i (for $i = 2, 3, \dots, k$). For each neighbor, we calculate the intersection I between our current candidate set and the authors of the next article (S_{next}):

$$I = C \cap Authors(a_i) \tag{2.4.2}$$

-
- **Case A (No overlap):** If $I = \emptyset$, the article a_i is ignored as it does not contain any of our primary candidates.
 - **Case B (Winner Found):** If $|I| = 1$, this single author is identified as the most consistent expert. **[STOP]**
 - **Case C (Subset Reduction):** If $|I| > 1$, we update $C \leftarrow I$ and continue to a_{i+1} to further narrow the tie.
4. **Step 4: Fallback (Stochastic Selection):** If the algorithm exhausts the top k neighbors and $|C| > 1$, a reviewer is selected from the remaining candidates $c \in C$ uniformly at random.

2.4.3 Pseudocode Representation

Listing 2.1: Hierarchical Candidate Filtering Algorithm

```
def find_reviewer(query_vector, article_database, k_depth=10):
    # 1. Retrieve ordered nearest neighbors
    nn = get_k_nearest_neighbors(query_vector, article_database, k=k_depth)

    # 2. Anchor candidates to the absolute closest match
    candidates = get_authors(nn[0])

    if len(candidates) == 1:
        return candidates[0]

    # 3. Disambiguation loop
    for i in range(1, k_depth):
        next_authors = get_authors(nn[i])
        overlap = candidates.intersection(next_authors)

        if len(overlap) == 1:
            return list(overlap)[0]
        elif len(overlap) > 1:
            candidates = overlap

    return random.choice(list(candidates))
```

2.4.4 Model Characteristics

This modeling approach possesses three defining characteristics:

- **Anchor Constraint:** The reviewer must be an author of the most relevant document (a_1). This ensures the reviewer’s expertise is directly tied to the closest matching work.

- **Frequency Priority:** It favors the author who appears most frequently within the local cluster of relevant papers.
- **Distance Decay:** It respects the ranking of neighbors; an author appearing in the 2nd neighbor takes precedence over one appearing in the 3rd.

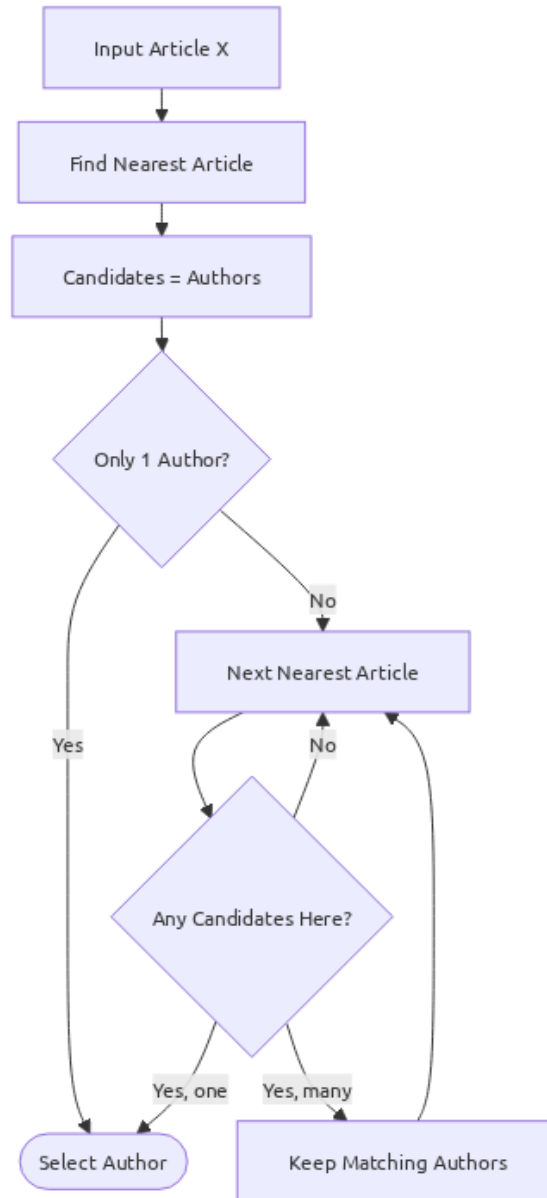


Figure 2.10: Simplified flow of the Search and Disambiguation logic.

It is important to note that this model is **representation-agnostic**. The logic holds true regardless of how the "Query Vector" q is generated. Whether the articles are represented via classical frequency methods or modern neural embeddings, the sequential disambiguation remains the core operational logic.

In the following section, we explore the different techniques used to transform the extracted abstracts and keywords into the numerical vectors required for this search process.

2.4.5 Feature Engineering and Modeling

To operationalize the sequential disambiguation model, the cleaned textual data must be transformed into a numerical format that allows for efficient similarity computation. Our approach focuses on a lexical representation that captures specific technical terminology through term frequency analysis.

Text Consolidation and Preprocessing

The primary features for each article are derived from two metadata fields: the *Arabic Abstract* and the *Keywords*. These fields are concatenated into a single `combined_text` string to ensure that both the contextual summary and the specific domain tags contribute to the document's signature.

For the lexical pipeline (TF-IDF), additional Arabic-specific preprocessing is applied:

- **Light Stemming:** Utilizing the *Tashaphyne* library, words are reduced to their base forms (e.g., stripping prefixes like "al-" and suffixes) to group morphological variants of the same root.
- **Stopword Removal:** Common Arabic functional words (e.g., "min", "ala", "fi") that do not contribute to semantic distinction are filtered out.

Lexical Vectorization (TF-IDF)

The preprocessed text is transformed into a high-dimensional vector space using the *Term Frequency-Inverse Document Frequency* (TF-IDF) algorithm. The vectorizer is configured with specific parameters to enhance retrieval quality:

- **Sublinear TF Scaling:** We apply $1 + \log(tf)$ scaling to dampen the influence of terms that appear very frequently in a single document.
- **Minimum Document Frequency ($min_df = 2$):** Terms that appear in only one document are ignored to reduce noise and prevent over-fitting to OCR errors or rare typos.
- **L2 Normalization:** Each document vector is normalized to unit length, ensuring that the length of the abstract does not disproportionately bias the similarity score.

Model Fitting and Inference Logic

The dataset is split into a **Training Set (80%)** and a **Testing Set (20%)**. The TF-IDF model is fitted exclusively on the training set to build a global vocabulary. At

inference time, a query article (from the test set or a new submission) is transformed into the same vector space.

The similarity between the query vector q and all training vectors d_i is calculated using the **Cosine Similarity** metric:

$$\text{sim}(q, d_i) = \frac{q \cdot d_i}{\|q\| \|d_i\|} \quad (2.4.3)$$

Once the similarity scores are computed, the system identifies the k nearest neighbors to execute the Sequential Disambiguation algorithm. As implemented in the pipeline, the system first "anchors" on the authors of the document with the highest similarity score. If a tie exists (multiple co-authors), it iteratively scans the subsequent neighbors in descending order of similarity, seeking the first candidate from the anchor set who reappears in the neighborhood. This ensures that the recommended reviewer is not only related to the closest topic but also possesses a consistent track record within the local thematic cluster.

2.5 Conclusion and Future Work

In this project, we successfully designed and implemented an intelligent article-reviewer recommendation system tailored for Arabic academic literature. By leveraging natural language processing (NLP) and content-based filtering, the system addresses the critical challenge of identifying suitable experts to review scientific manuscripts based on the semantic alignment between the manuscript's content and the author's historical research output.

2.5.1 Summary of Methodology

The core of the system relies on the construction of **Author Expertise Profiles**. By aggregating the abstracts and keywords of all articles associated with a specific researcher, we transformed individual authors into high-dimensional vectors within a semantic space. We utilized:

- **Text Preprocessing:** Specialized Arabic NLP techniques, including normalization and stop-word removal, to handle the linguistic complexities of the dataset.
- **Vectorization:** A TF-IDF (Term Frequency-Inverse Document Frequency) approach to identify the unique "lexical signature" of each researcher.
- **Similarity Measurement:** Cosine Similarity to rank the closest matching reviewers for any given new submission.

2.5.2 Addressing Dataset Limitations

It is important to note that the dataset used for this implementation featured randomly assigned authors. While this prevents the model from learning from "real-world" historical bias, it served as a robust proof-of-concept for a **cold-start recommendation engine**. The results demonstrate that even without historical peer-review data, the system can accurately identify subject matter experts solely through the linguistic overlap of their published research and the new submission.

2.5.3 Key Findings

The system demonstrated a high degree of precision in mapping niche technical terms to the relevant authors. By combining both keywords and abstracts, the model captured both the broad domain (via keywords) and the specific context (via abstracts), leading to more nuanced recommendations than a simple keyword search would allow.

2.5.4 Future Work

Moving forward, several enhancements can be made to increase the system's accuracy and utility:

- **Deep Learning Integration:** Transitioning from TF-IDF to Arabic-specific transformer models like *AraBERT* to capture deep semantic meanings and synonyms.
- **Reviewer Workload Balancing:** Integrating an optimization layer that considers the number of articles an author is currently reviewing to prevent reviewer fatigue.
- **Hybrid Modeling:** If real-world reviewer history becomes available, combining this content-based approach with Collaborative Filtering to account for reviewer reliability and quality.

In conclusion, this system provides a scalable, objective, and efficient solution for academic journals and conference organizers, significantly reducing the manual effort required in the editorial process.